

# ADVANCE REACT JS

SUNITA D RATHOD

Functional components- Refs, Use effects, Hooks,  
Flow architecture, Model-View- Controller framework, Flux,  
Bundling the application. Web pack.



# WHAT IS A HOOKS?

Hooks allow function components to have access to state and other React features.

Because of this, class components are generally no longer needed.

Although Hooks generally replace class components, there are no plans to remove classes from React.



# WHAT IS A HOOKS?

Hooks allow us to "hook" into React features such as state and lifecycle methods.

There are 3 rules for hooks:

- Hooks can only be called inside React function components.
- Hooks can only be called at the top level of a component.
- Hooks cannot be conditional



# React `useState` Hook

The React `useState` Hook allows us to track state in a function component.

State generally refers to data or properties that need to be tracking in an application.

## Import `useState`

To use the `useState` Hook, we first need to `import` it into our component.



# React `useState` Hook

## Initialize `useState`

We initialize our state by calling `useState` in our function component.

`useState` accepts an initial state and returns two values:

- The current state.
- A function that updates the state.



# React `useState` Hook

## Example:

Initialize state at the top of the function component.

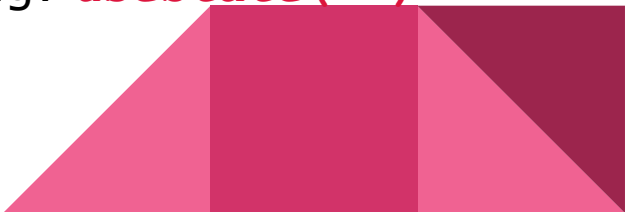
```
import { useState } from "react";

function FavoriteColor() {
  const [color, setColor] = useState("");
}
```

The first value, **color**, is our current state.

The second value, **setColor**, is the function that is used to update our state.

we set the initial state to an empty string: **useState("")**



## Read State

We can now include our state anywhere in our component.

### Example:

Use the state variable in the rendered component.

```
import { useState } from "react";
import ReactDOM from "react-dom/client";

function FavoriteColor() {
  const [color, setColor] = useState("red");

  return <h1>My favorite color is {color}!</h1>
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<FavoriteColor />);
```



# Update State

To update our state, we use our state updater function.

Use a button to update the state:

```
import { useState } from "react";
import ReactDOM from "react-dom/client";

function FavoriteColor() {
  const [color, setColor] = useState("red");

  return (
    <>
      <h1>My favorite color is {color}!</h1>
      <button
        type="button"
        onClick={() => setColor("blue")}
      >Blue</button>
    </>
  )
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<FavoriteColor />);
```

# What Can State Hold

The `useState` Hook can be used to keep track of strings, numbers, booleans, arrays, objects, and any combination of these!

We could create multiple state Hooks to track individual values.



```
import { useState } from "react";
import ReactDOM from "react-dom/client";

function Car() {
  const [brand, setBrand] = useState("Ford");
  const [model, setModel] = useState("Mustang");
  const [year, setYear] = useState("1964");
  const [color, setColor] = useState("red");

  return (
    <>
      <h1>My {brand}</h1>
      <p>
        It is a {color} {model} from {year}.
      </p>
    </>
  )
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
```

# My Ford

It is a red Mustang from 1964

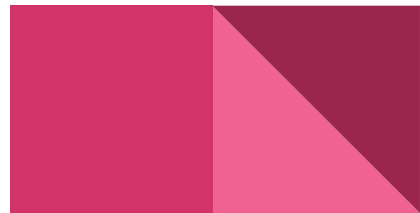
```
import { useState } from "react";
import ReactDOM from "react-dom/client";

function Car() {
  const [car, setCar] = useState({
    brand: "Ford",
    model: "Mustang",
    year: "1964",
    color: "red"
  });

  return (
    <>
      <h1>My {car.brand}</h1>
      <p>
        It is a {car.color} {car.model} from {car.year}.
      </p>
    </>
  )
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
```

Or, we can just use one state and include an object instead!



# React **useEffect** Hooks

The **useEffect** Hook allows you to perform side effects in your components.

Some examples of side effects are: fetching data, directly updating the DOM, and timers.

**useEffect** accepts two arguments. The second argument is optional.

**useEffect** (<function>, <dependency>)



Use `setTimeout()` to count 1 second after initial render:

```
import { useState, useEffect } from "react";  
import ReactDOM from "react-dom/client";
```

```
function Timer() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    setTimeout(() => {  
      setCount((count) => count + 1);  
    }, 1000);  
  });  
  
  return <h1>I've rendered {count} times!</h1>;  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Timer />);
```

**useEffect** runs on every render. That means that when the count changes, a render happens, which then triggers another effect.

There are several ways to control when side effects run.

We should always include the second parameter which accepts an array. We can optionally pass dependencies to **useEffect** in this array.



## 1. No dependency passed:

```
useEffect(() => {  
    //Runs on every render  
});
```

## 2. An empty array:

```
useEffect(() => {  
    //Runs only on the first render  
}, []);
```

## 3. Props or state values:

```
useEffect(() => {  
    //Runs on the first render  
    //And any time any  
    dependency value changes  
}, [prop, state]);
```





Here is an example of a `useEffect` Hook that is dependent on a variable. If the `count` variable updates, the effect will run again:

Count: 5



Calculation: 10

```
import { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";

function Counter() {
  const [count, setCount] = useState(0);
  const [calculation, setCalculation] = useState(0);

  useEffect(() => {
    setCalculation(() => count * 2);
  }, [count]); // <- add the count variable here

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>+</button>
      <p>Calculation: {calculation}</p>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Counter />);
```

# React useContext Hook

React Context is a way to manage state globally.

It can be used together with the `useState` Hook to share state between deeply nested components more easily than with `useState` alone.



# React `useRef` Hook

The `useRef` Hook allows you to persist values between renders.

It can be used to store a mutable value that does not cause a re-render when updated.

It can be used to access a DOM element directly.

If we tried to count how many times our application renders using the `useState` Hook, we would be caught in an infinite loop since this Hook itself causes a re-render.

To avoid this, we can use the `useRef` Hook.



# React **useRef** Hook

Use **useRef** to track application renders.

Render Count: 1

```
import { useState, useEffect, useRef } from "react";
import ReactDOM from "react-dom/client";

function App() {
  const [inputValue, setInputValue] = useState("");
  const count = useRef(0);

  useEffect(() => {
    count.current = count.current + 1;
  });

  return (
    <>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h1>Render Count: {count.current}</h1>
    </>
  );
}
```

`useRef()` only returns one item. It returns an Object called `current`.

When we initialize `useRef` we set the initial value: `useRef(0)`.

It's like doing this: `const count = {current: 0}`. We can access the count by using `count.current`.



## Accessing DOM Elements

In general, we want to let React handle all DOM manipulation.

But there are some instances where **useRef** can be used without causing issues.

In React, we can add a **ref** attribute to an element to access it directly in the DOM.



Use `useRef` to focus the input:

```
import { useRef } from "react";  
import ReactDOM from "react-dom/client";
```

```
function App() {  
  const inputElement = useRef();  
  
  const focusInput = () => {  
    inputElement.current.focus();  
  };  
  
  return (  
    <>  
      <input type="text" ref={inputElement} />  
      <button onClick={focusInput}>Focus Input</button>  
    </>  
  );  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<App />);
```

OUTPUT

 Focus Input

# Tracking State Changes

The `useRef` Hook can also be used to keep track of previous state values.

This is because we are able to persist `useRef` values between renders.





Use `useRef` to keep track of previous state values:

**Current Value: ABCD**

**Previous Value: ABC**

```
import { useState, useEffect, useRef } from "react";
import ReactDOM from "react-dom/client";

function App() {
  const [inputValue, setInputValue] = useState("");
  const previousInputValue = useRef("");

  useEffect(() => {
    previousInputValue.current = inputValue;
  }, [inputValue]);

  return (
    <>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h2>Current Value: {inputValue}</h2>
      <h2>Previous Value: {previousInputValue.current}</h2>
    </>
  );
}
```

